

Acesso a Banco de Dados SQL

Objetivo da Aula

Compreender a configuração e o gerenciamento de dados no MySQL e utilizar o Sequelize para mapeamento objeto-relacional no NodeJS, incluindo o uso de migrations e seeders.

Apresentação

A persistência em banco de dados é uma necessidade básica para a criação de sistemas cadastrais, e devemos compreender como o NodeJS lida com uma base de dados. Embora a linguagem SQL fosse o meio padrão para acessar e manipular dados em sistemas antigos, hoje é muito comum o uso de ferramentas para mapeamento objeto-relacional, como o Sequelize.

Antes de começar a trabalhar com o Sequelize, será necessário ter um banco de dados, e aqui veremos como instalar e configurar o MySQL. Existem diversas opções de banco de dados, como Oracle, SQL Server e Postgres, mas o MySQL tem um gerenciamento bastante simples, além de boa eficiência, figurando-se entre as opções mais adotadas pelas empresas.

Além de utilizar o Sequelize para o gerenciamento dos dados, é possível criar migrations e seeders, ferramentas que irão possibilitar a criação e carga inicial do banco de dados a partir do NodeJS.

1. Instalação e Configuração do MySQL

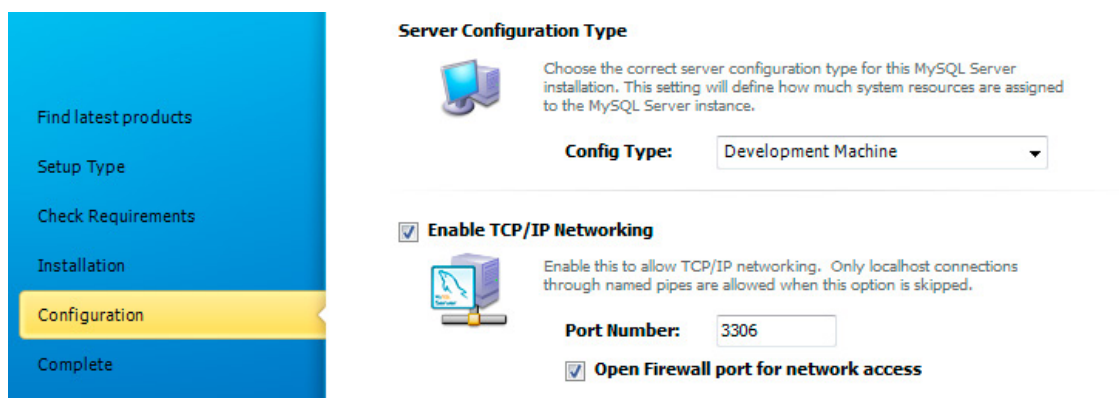
O banco de dados **MySQL** segue o modelo relacional, utiliza a sintaxe SQL e oferece uma versão gratuita, ou **community**, para desenvolvimento. Quando o MySQL foi comprado pela Oracle, ele passou a ter versões pagas, causando a migração de muitos desenvolvedores para o **MariaDB**.

Para instalar o servidor **MySQL** no ambiente Windows, você deverá baixar o instalador a partir do endereço **<https://dev.mysql.com/downloads/installer>**. Em seguida, execute o instalador e siga os passos indicados pelo assistente. Outros SOs também podem ser utilizados, incluindo o WSL, se for o caso.

Vamos utilizar a instalação completa, ou **Full**, com o **MySQL Workbench**, que é o gerenciador visual padrão para o banco de dados MySQL. Durante a instalação, será solicitada a senha do **root**, que é o usuário principal do banco, e não podemos nos esquecer de escolher o modo de execução como **serviço**, pois irá permitir iniciar ou fechar o banco através do gerenciador de tarefas.

Um passo importante na instalação é a configuração de rede, em que deve ser habilitada a conexão **TCP/IP**, utilizando a porta padrão **3306**. O próprio instalador oferece a opção de abertura de portas no firewall.

Figura 1: Configuração de rede para o MySQL



Fonte: Elaboração própria.

Terminada a instalação, verifique se o serviço do MySQL está ativo, através da aba de serviços do gerenciador de tarefas, e abra o **MySQL Workbench** para começar a definir as estruturas do banco de dados.

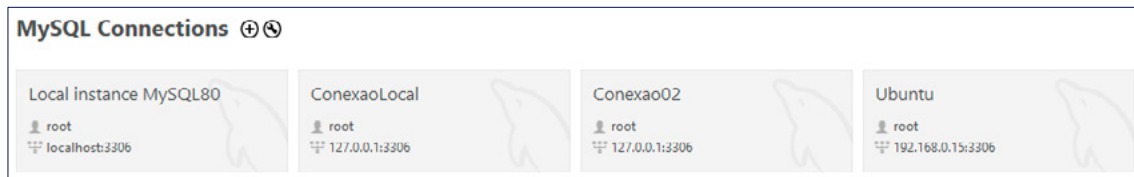


Você Sabia?

Uma boa opção para o gerenciamento de banco de dados é o **DBeaver**, criado na plataforma **Java**. Além de oferecer suporte a qualquer tipo de banco de dados do mercado, os drivers são baixados por demanda.

Normalmente, o MySQL Workbench já oferece uma conexão configurada para o MySQL local, mas, se não tiver conexões, deve-se criar uma através do botão de **adição**, utilizando endereço **localhost**, porta **3306**, usuário **root** e senha escolhida.

Figura 2: Conexão com o MySQL



Fonte: Elaboração própria.

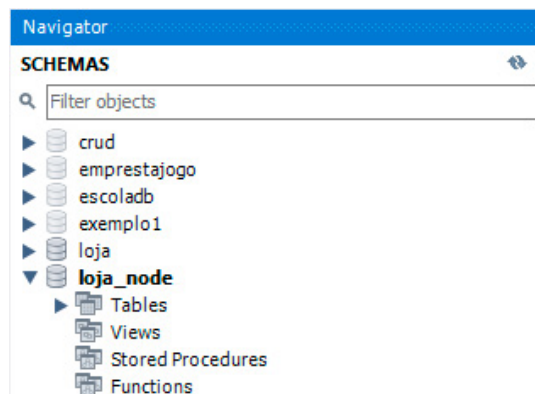
Após conectar com o servidor, inicie uma aba para execução de comandos SQL, através do menu **File**, seguido de **New Query Tab**, podendo também ser utilizado o **CTRL+T** ou o primeiro ícone da toolbar. Com a aba aberta, digite os comandos da listagem seguinte para criação do banco de dados e da tabela de **produtos**, que utilizaremos em nosso exemplo.

```
create database loja_node;
use loja_node;
create table produtos(
    codigo int not null primary key auto_increment,
    nome varchar(50) not null,
    quantidade int null);
```

O primeiro comando cria o banco de dados **loja_node**, o segundo requisita o uso do banco, e o terceiro cria a tabela **produtos**. Para a tabela produtos foram definidos os campos **código** e chave primária e foram incrementados, automaticamente, **nome**, do tipo texto, e **quantidade**, numérico.

Você precisará selecionar cada comando e clicar no botão de execução, que tem um raio desenhado. Após executar os comandos, verifique o banco criado, no **painel de navegação**, exibido à esquerda no ambiente, sendo necessário clicar no botão de **refresh**, posicionado no canto superior direito do painel.

Figura 3: Banco de dados criado



Fonte: Elaboração própria.

Para inserir alguns registros, pode ser feito um processo similar, abrindo a aba para execução de SQL e utilizando os comandos apresentados a seguir.

```
use loja_node;

insert into produtos (nome,quantidade)

values ("Laranja",500),

("Banana",1000);
```

O primeiro comando requisita o uso do banco **loja_node**, e o segundo insere dois registros na tabela **produtos**. Note que temos apenas os campos nome e quantidade, já que o valor do código é gerado automaticamente.

Após inserir, utilize o comando a seguir, ainda na aba de execução de SQL.

```
select * from produtos;
```

O resultado da consulta é disponibilizado na parte inferior da janela, como pode ser observado na figura seguinte.

Figura 4: Resultado da consulta SQL

The image shows a 'Result Grid' window with a toolbar at the top containing icons for 'Filter Rows', 'Edit', 'Export/Import', and 'Wrap Cell Content'. Below the toolbar is a table with the following data:

	codigo	nome	quantidade
▶	1	Laranja	500
	2	Banana	1000
*	NULL	NULL	NULL

Fonte: Elaboração própria.



Para Praticar

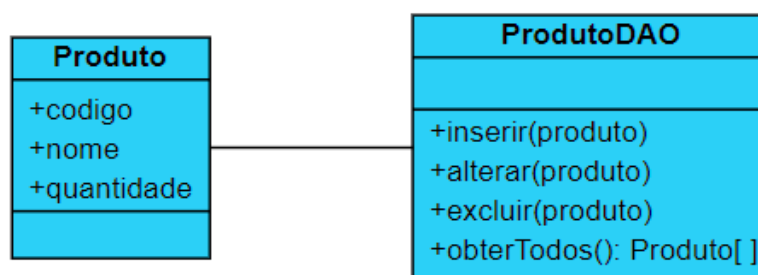
Além de permitir gerenciar a base de dados por meio de SQL, o MySQL Workbench oferece diversos recursos de gerenciamento, acionados por botões ou clique do botão direito. Explore sua interface e experimente as diversas funcionalidades do gerenciador para ganho de produtividade.

2. Características Gerais do ORM Sequelize

Antigamente, os sistemas com persistência em banco de dados relacional tinham comandos SQL espalhados por todo o código, criados como variáveis de texto que acabavam se confundindo com a sintaxe da plataforma. Pior que isso, era comum o uso de concatenação de texto nos comandos SQL, sujeitando o aplicativo a ataques do tipo **SQL Injection**, o que acabou sendo resolvido com a utilização de comandos parametrizados.

Com a grande dificuldade de manutenção, programadores passaram a adotar o padrão de desenvolvimento **Data Access Object (DAO)**, que define classes de entidade, para representar as tabelas, e classes DAO, para concentrar todas as operações sobre o banco de dados. Em termos práticos, quando a consulta é efetuada, cada registro obtido é transformado em uma instância de entidade, fazendo com que o restante do sistema trabalhe apenas com objetos.

Figura 5: Padrão DAO aplicado a Produto



Fonte: Elaboração própria.

A metodologia de conversão de registros para objetos ficou conhecida como “mapeamento objeto-relacional”, ou **ORM**, e, como os comandos SQL são muito similares e as tarefas de implementação são repetitivas, diversos **frameworks** foram criados para automatizar esse

tipo de desenvolvimento. Entre as opções que se tornaram populares, podemos destacar o Entity Framework, utilizado no dotNet, JPA, da plataforma Java, SQL Alchemy, no Python, Eloquent (Laravel), utilizado no PHP, e **Sequelize**, para o NodeJS.

O framework Sequelize permite definir o mapeamento objeto-relacional de forma simples, sem a necessidade de comandos SQL. Ele trabalha com **MySQL**, Postgres, MariaDB, Oracle, SQL Server e muitas outras opções de mercado.

Segundo a arquitetura definida no framework, um objeto **Sequelize** efetua a conexão com o banco de dados, enquanto **Model** define as entidades. As operações para gerenciamento das entidades são baseadas em **promise**, ou seja, executam de forma assíncrona, e o formato de dados utilizado é **JSON**.

Para utilizar o Sequelize com acesso ao MySQL, em seu projeto, você precisa incluir as bibliotecas **Sequelize** e **mysql2**.

```
npm install sequelize mysql2
```

Após instalar as bibliotecas, você precisará definir a conexão com o banco, de acordo com o código seguinte, no arquivo **db.js**. Note que é utilizado import, exigindo definir type como module em package.json.

```
import { Sequelize } from "sequelize";  
  
const sequelize = new Sequelize("loja_node", "root", "adm12",  
    {dialect: "mysql", host: "127.0.0.1"});  
  
export default sequelize;
```

Observando o código, temos a geração de uma instância de Sequelize, que representa a conexão com o banco de dados, tendo como parâmetros o **nome** do banco, o **usuário** e a **senha**, além de uma parametrização em JSON, informando o dialeto utilizado (MySQL) e o endereço do servidor. Ao final, o objeto de conexão é exportado, para que seja utilizado no resto do sistema.

O passo seguinte é a definição da entidade no arquivo **model-produto.js**.

```
import { Sequelize } from "sequelize";
import db from "../db.js"; // Objeto sequelize

export default db.define("produto", {
  codigo: {type: Sequelize.INTEGER.UNSIGNED, primaryKey: true,
    autoIncrement: true, allowNull: false},
  nome: {type: Sequelize.STRING, allowNull: false},
  quantidade: {type: Sequelize.INTEGER, allowNull: true}
}, {timestamps: false}); // createdAt, updatedAt
```

A entidade é criada por meio do método `define`, sendo exportada por padrão, e tem como parâmetros o nome da entidade e a caracterização de seus campos no formato JSON. Aqui, temos os campos `codigo`, `nome` e `quantidade`, que foram configurados para se adequar à tabela `produtos`, criada anteriormente.

O último parâmetro define opções específicas do Sequelize, que, em nosso caso, foi utilizado para remover os marcadores de tempo (`timestamp`). Segundo o comportamento padrão do Sequelize, para cada entidade teríamos a criação dos campos **`createdAt`** e **`updatedAt`**, para guardar o instante em que a entidade foi criada e o momento de sua última modificação, mas a opção **`timestamp`**, com valor **`false`**, elimina esse comportamento.

Com a entidade definida, podemos criar um controlador no padrão DAO, que pode ser observado na implementação parcial da listagem seguinte.

```
import ProdutoRepo from "../model-produto.js";

export async function obterTodos() {
  return await ProdutoRepo.findAll();
}

export async function obterProduto(codigo) {
  return await ProdutoRepo.findByPk(codigo);
}
```

```
export async function criarProduto(nome, quantidade) {
  const produto = await ProdutoRepo.create(
    {nome: nome, quantidade: quantidade});
  return produto.codigo;
}
```

Na função **obterTodos**, temos o retorno de todos os produtos da base de dados, através da chamada do método **findAll**, enquanto **obterProduto** retorna apenas o produto cujo código foi fornecido no parâmetro, com a utilização do método **findByPk**. A função **criarProduto** recebe o nome e a quantidade e cria um produto na base de dados, através do método **create**, retornando o código do novo produto, que é gerado, automaticamente, ao final.

O código apresentado a seguir demonstra o uso da consulta.

```
import {obterTodos} from "../controller-produto.js";
obterTodos().then((x)=>console.log(JSON.stringify(x)));
```

Como a chamada é assíncrona, você precisa de uma callback para tratar o retorno – que, no exemplo, é impresso no console. O resultado da execução pode ser observado a seguir.

Figura 6: Consulta efetuada via Sequelize

```
PS D:\NodeProjs\dback\unidade2\Exemplo03> node index1
[{"codigo":1,"nome":"Laranja","quantidade":500}, {"codigo":2,"nome":"Banana","quantidade":1001}, {"codigo":4,"nome":"Manga","quantidade":0}, {"codigo":7,"nome":"Abacate","quantidade":0}]
```

Fonte: Elaboração própria.

3. Configuração do Banco com Migrations e Seeders

Uma forma mais organizada de utilizar o Sequelize é por meio do cliente de linha de comando ou **Sequelize CLI**. Através dele, é possível definir a estrutura de persistência do projeto e a conexão com o banco, gerar o código para as entidades e, até mesmo, criar o banco e as tabelas, além de carregar os dados, a partir do código em JavaScript.

Crie um diretório vazio, abra no VS Code e configure o projeto, utilizando os comandos apresentados a seguir.


```
npm init -y
npm install sequelize mysql2
npm install --save-dev sequelize-cli
npx sequelize-cli init
```

Como podemos observar, são instaladas as dependências necessárias e, ao final, é executado o comando **init**, de **sequelize-cli**, por meio do **npx**. A estrutura necessária é definida através desse comando, utilizando os diretórios **config**, **migrations**, **models** e **seeders**.

Você verá as três opções para conexão (development, test e production) no arquivo **config.json**, que fica no diretório **config**. Modificando as características de cada uma, podemos definir qual banco será usado no desenvolvimento, utilizado como padrão, testes ou produção.

Altere a configuração do banco de desenvolvimento, conforme a listagem.

```
{
  "development": {
    "username": "root", "password": "adm12",
    "database": "pessoal_dev", "host": "127.0.0.1",
    "dialect": "mysql"
  },
```

Como pode ser observado, será necessário informar usuário, senha, nome do banco, endereço do servidor e dialeto. Após efetuar a modificação, salve o arquivo e execute o comando a seguir para **criar o banco de dados** no MySQL.

```
npx sequelize-cli db:create
```

Agora, vamos definir as nossas entidades, através dos comandos seguintes.

```
npx sequelize-cli model:generate --name Pessoa
  --attributes nome:string,sobrenome:string,telefone:string
```

```
npx sequelize-cli model:generate --name Dependente
  --attributes nome:string,pessoaId:integer
```

As entidades serão criadas no diretório **models**, utilizando uma programação bastante robusta, devido ao uso do framework. Também teremos o código para geração de cada tabela no diretório **migrations**.

Aqui foi definida a entidade Pessoa, com os campos nome, sobrenome e telefone, e a entidade Dependente, com os campos nome e pessoaId. Além dos campos especificados, teremos os campos **id**, **createdAt** e **updatedAt** sendo gerados automaticamente para cada uma das entidades.

Não valeria a pena analisar todo o código gerado pelo Sequelize CLI, mas é necessário compreender como são definidos os relacionamentos. Vamos definir um relacionamento de 1:N entre Pessoa e Dependente, alterando o código de cada uma das entidades.

No arquivo **dependente.js**, altere o método estático **associate**, de acordo com o fragmento de código apresentado a seguir.

```
static associate(models) {
  Dependente.belongsTo(models.Pessoa, {
    foreignKey: 'pessoaId',
    onDelete: 'CASCADE'
  })
}
```

Já para o arquivo **pessoa.js**, utilize a configuração apresentada a seguir.

```
static associate(models) {
  Pessoa.hasMany(models.Dependente, {
    foreignKey: 'pessoaId',
  })
}
```

Observando as modificações, você verá que a tabela de **dependentes** terá uma chave estrangeira, no campo **peessoald**, apontando para a chave primária da tabela de **pessoas**, devido ao uso de **belongsToMany**, exigindo a configuração da cardinalidade na entidade Pessoa, através do método **hasMany**. Note, também, que foi definido o padrão cascade para a deleção, ou seja, todos os dependentes são removidos ao deletar uma pessoa.

Também é necessário incluir o relacionamento no arquivo de criação para a tabela dependentes, que tem o nome **create-dependente.js** e um prefixo com a data e hora da geração, no diretório **migrations**, de acordo com o fragmento de código apresentado a seguir, em que temos a configuração do campo **peessoald**.

```
peessoaId: {
  type: Sequelize.INTEGER,
  onDelete: 'CASCADE',
  references: {model: 'Pessoas', key: 'id', as: 'peessoaId'}
},
```

Finalizadas as alterações, vamos gerar as tabelas a partir dos arquivos de **migrations**, utilizando o comando apresentado a seguir.

```
npx sequelize-cli db:migrate
```



Destaque

Você poderá verificar a criação do banco e das tabelas, bem como a definição do relacionamento entre as tabelas, através do MySQL Workbench.

Não são raras as situações em que temos de alimentar o banco com alguns dados iniciais – como a carga de cidades e estados em um sistema cadastral –, e uma forma de resolver isso é através dos **seeders**. Eles permitem o uso de JSON para definir as entidades que serão incluídas no banco de dados, lembrando que os campos com geração automática não podem ser controlados.

Utilize os comandos apresentados a seguir para gerar o seeder de Pessoa.

```
npx sequelize-cli seed:generate --name pessoa
```

O arquivo **pessoa.js**, com prefixo indicando data e hora da geração, será criado no diretório **seeders**. Altere seu conteúdo, de acordo com a listagem que é apresentada a seguir.

```
'use strict';

/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up (queryInterface, Sequelize) {
    return queryInterface.bulkInsert('Pessoas', [{
      nome: 'John', sobrenome: 'Doe', telefone: '21-21212121',
      createdAt: new Date(), updatedAt: new Date()
    }], {});
  },
  async down (queryInterface, Sequelize) {
    return queryInterface.bulkDelete('Pessoas', null, {});
  }
};
```

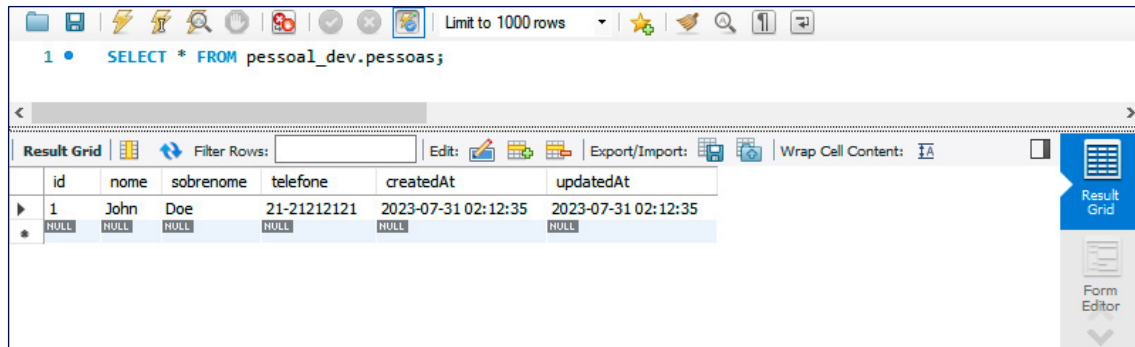
Através do método **bulkInsert**, podemos incluir os dados de várias pessoas, por meio de um vetor de objetos JSON. Deve-se informar os dados para os campos que não são gerados automaticamente, além de preencher os campos **createdAt** e **updatedAt**, utilizados para log de inserção e alteração.

Após efetuar, salve as modificações no arquivo e execute os seeders através do comando apresentado a seguir.

```
npx sequelize-cli db:seed:all
```

Agora, é só abrir o SQL Workbench e verificar os dados inseridos.

Figura 7: Dados inseridos via seeder



The screenshot shows the MySQL Workbench interface. At the top, a query is entered: `SELECT * FROM pessoal_dev.pessoas;`. Below the query, the 'Result Grid' is displayed, showing a single row of data. The columns are: id, nome, sobrenome, telefone, createdAt, and updatedAt. The data in the row is: 1, John, Doe, 21-21212121, 2023-07-31 02:12:35, and 2023-07-31 02:12:35. The interface also includes a toolbar with various icons and a sidebar with 'Result Grid' and 'Form Editor' buttons.

id	nome	sobrenome	telefone	createdAt	updatedAt
1	John	Doe	21-21212121	2023-07-31 02:12:35	2023-07-31 02:12:35

Fonte: Elaboração própria.

Considerações Finais da Aula

Com o objetivo de gerenciar uma base de dados via NodeJS, iniciamos com a instalação e configuração do MySQL. Para as tarefas de configuração, o uso do MySQL Workbench foi um grande facilitador, e uma outra boa opção seria o DBeaver, implementado em Java, com acesso para qualquer tipo de banco de dados e download de drivers sob demanda.

Após instalar o banco de dados, vimos como o Sequelize funciona, criando manualmente os componentes necessários para o gerenciamento da base de dados via NodeJS. Em seguida, pudemos verificar um processo similar, com a utilização do Sequelize CLI, a partir da criação das estruturas do Sequelize de uma forma mais organizada.

Por fim, pudemos observar como o Sequelize CLI, além de simplificar o uso do mapeamento objeto-relacional, pode ser utilizado para gerar migrations e seeders – elementos que permitem criar as estruturas do banco e efetuar a carga inicial, a partir da plataforma NodeJS.

Materiais Complementares



Artigo

MySQL Workbench Manual

2023, MySQL Workbench.

Esse é um extenso artigo, com vários links internos, demonstrando todas as funcionalidades do MySQL Workbench.

Link para acesso: <https://dev.mysql.com/doc/workbench/en/> (acesso em 12 set. 2023.)

 Artigo**Migrations**

2023, Zoé. Sequelize.

O artigo traz uma descrição completa de migrations e seeders, para criação do banco e da alimentação inicial. Também descreve, em detalhes, os comandos para aplicar ou reverter a ação desses elementos.

Link para acesso: <https://sequelize.org/docs/v6/other-topics/migrations/> (acesso em 12 set. 2023.)

Referências

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92164?guid=1691771378051&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691771378051%26quantidadePaginas%3d1%26codigoRegistro%3d92164%2392164&i=8>. Acesso em: 12 set. 2023.

MANZANO, J. **MySQL 5.5 interativo: guia essencial de orientação e desenvolvimento**. São Paulo: Érica, 2011. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92472?guid=1692685371962&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1692685371962%26quantidadePaginas%3d1%26codigoRegistro%3d92472%2392472&i=6>. Acesso em: 12 set. 2023.

OLIVEIRA, C.; ZANETTI, H. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92165?guid=1691773379732&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691773379732%26quantidadePaginas%3d1%26codigoRegistro%3d92165%2392165&i=24>. Acesso em: 12 set. 2023.

OLIVEIRA, C.; ZANETTI, H. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92479?guid=1691770593236&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691770593236%26quantidadePaginas%3d1%26codigoRegistro%3d92479%2392479&i=4>. Acesso em: 12 set. 2023.